

DevOps Lab App

Aplicación web mínima (**Node.js + TypeScript + Express**) usada como caso práctico para el laboratorio técnico de la Actividad 3: **configurar un pipeline CI/CD básico usando GitHub Actions y Jenkins**.

Endpoints

Método	Ruta	Descripción
GET	/health	Health check, responde { "status": "ok" }
GET	/api/greet?name=X	Devuelve un saludo personalizado
POST	/api/sum	Recibe { "a": number, "b": number } y devuelve la suma

Ejecutar en local

```
npm install
npm run typecheck    # verificación de tipos (tsc --noEmit)
npm run lint         # ESLint (reglas TS)
npm test             # pruebas con Jest + ts-jest + cobertura
npm run build        # compila TypeScript a dist/
npm start            # corre dist/index.js en http://localhost:3000
npm run dev          # alternativa: corre src/index.ts directo con ts-node
```

Flujo CI/CD

CI — GitHub Actions (.github/workflows/ci.yml)

Se dispara automáticamente en cada push y en cada pull_request hacia main. Etapas:

1. Checkout del código (actions/checkout).
2. Configuración de Node.js con caché de dependencias.
3. Instalación de dependencias (npm ci).
4. Verificación de tipos (npm run typecheck).
5. Análisis estático con ESLint (npm run lint).
6. Ejecución de pruebas con Jest (npm test) y publicación del reporte de cobertura como artefacto.
7. Compilación a JavaScript (npm run build), validando que el proyecto compile antes de construir la imagen.

Si cualquier paso falla, el workflow se marca en rojo y bloquea la confianza en ese commit/PR.

CD — Jenkins (Jenkinsfile)

Pipeline declarativo con los stages pedidos por la guía:

1. **Clonar el repositorio** (checkout scm).
2. **Construir la imagen Docker**: el Dockerfile es multi-stage — la primera etapa instala dependencias y compila TypeScript (npm run build); la segunda copia solo dist/ y las dependencias de producción, corriendo como usuario no root.
3. **Escaneo básico de la imagen** (verifica que la imagen se construyó correctamente; punto de extensión para herramientas como Trivy).

4. **Publicar la imagen en un registro — GitHub Container Registry (GHCR)**, usando credenciales guardadas en Jenkins (ghcr-credentials: usuario de GitHub + Personal Access Token con permiso write:packages).

No es necesario usar DockerHub: la guía acepta “DockerHub o similar”, y GHCR evita crear una cuenta adicional porque reutiliza tu cuenta de GitHub.

Configuración necesaria antes de correr el Jenkinsfile

1. En GitHub: **Settings → Developer settings → Personal access tokens** → generar un token con el permiso write:packages (y read:packages).
2. En Jenkins: **Manage Jenkins → Credentials** → agregar una credencial tipo *Username with password*:
 - Username: tu usuario de GitHub.
 - Password: el token generado en el paso 1.
 - ID: ghcr-credentials (debe coincidir con el credentialsId del Jenkinsfile).
3. En el Jenkinsfile, reemplazar TU_USUARIO_GITHUB por tu usuario u organización real de GitHub.
4. Asegurarse de que el agente de Jenkins tenga Docker instalado y el usuario jenkins tenga permisos para usarlo (usermod -aG docker jenkins).

Estructura del repositorio

```
.
├── .github/workflows/ci.yml    # Pipeline CI (GitHub Actions)
├── Jenkinsfile                # Pipeline CD (Jenkins)
├── Dockerfile                 # Multi-stage: build TS → imagen de producción
├── tsconfig.json
├── jest.config.js
├── src/index.ts               # Código de la aplicación (TypeScript)
├── tests/app.test.ts          # Pruebas unitarias (Jest + ts-jest + Supertest)
└── package.json
```